



LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

Lesson 7

Replication & Scaling MySQL



Biên soạn bởi Hội đồng chuyên môn - Domain Backend



GIẢNG VIÊN



Phan Hồng Linh

Project Manager

Data & Solution Architect Department

MỤC LỤC



01. Data System Overview

02. Replication MySQL

03. Scaling MySQL

1

**Data system
Overview**

Các vấn đề về data system



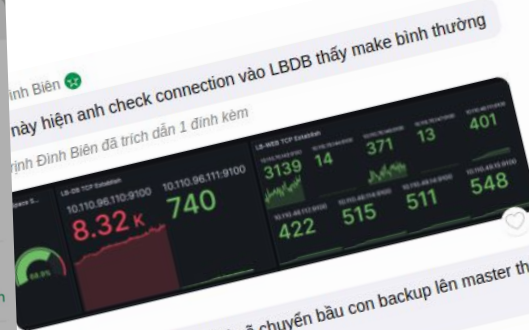
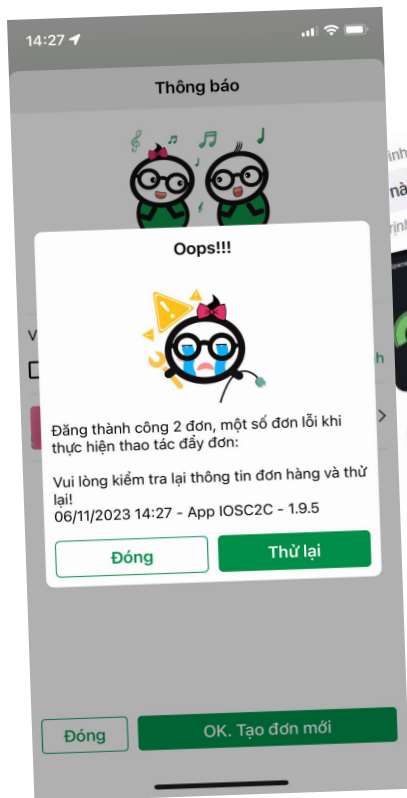
Các yếu tố khi xây dựng hệ thống data system

"The Internet was done so well that most people think of it as a natural resource like the Pacific Ocean, rather than something that was man-made. When was the last time a thing like that was so error-free?"

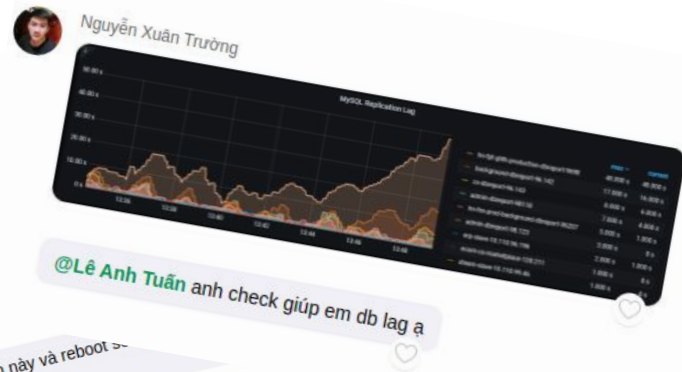
- Alan Kay -



Data system problem



nên anh để giờ thấp tải thì sẽ chuyển bầu con backup lên master thay cho con này và reboot zombie trên node nhé.



@Lê Anh Tuấn anh check giúp em db lag ạ



Ví MoMo

<https://momo.vn> > hoi-dap > tai-khoan-bi-tru-tien-nhu...

Tài khoản bị trừ tiền nhưng giao dịch đang chờ xử lý

Các trường hợp giao dịch nạp/rút tiền có tình trạng "đang chờ xử lý" xảy ra do dữ liệu giữa MoMo và ngân hàng chưa đồng bộ kịp thời. Khi gặp trường hợp này ...



Bảo Đại Đoàn Kết

<http://daidoanket.vn> > Khoa học

Tiền bỗng bay hơi ở ví Momo

7 thg 8, 2021 — Liên tiếp trong thời gian gần đây, nhiều khách hàng phản ánh việc dùng ví điện tử Momo bị mất tiền, mặc dù không cung cấp mã bảo mật (OTP) ...



Các yếu tố khi xây dựng data system

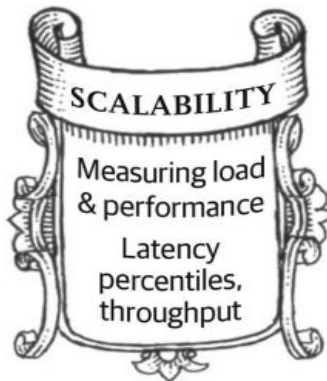


Hệ thống phải hoạt động chính xác ngay cả khi có lỗi phần mềm hoặc phần cứng, thậm chí là lỗi của con người.

Khi hệ thống phát triển (về khối lượng dữ liệu, lưu lượng truy cập hoặc độ phức tạp), cần có phương án và khả năng để xử lý sự tăng trưởng đó.

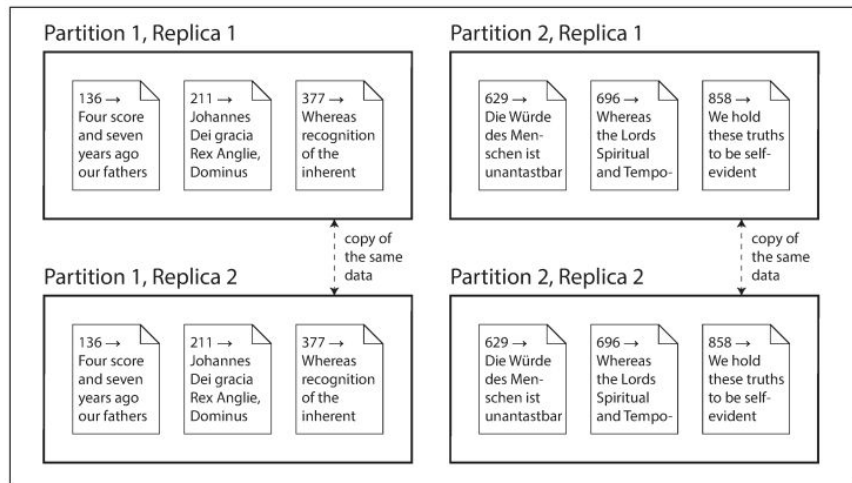


Chỉnh sửa, thay đổi cấu hình, dễ dàng vận hành. Nhiều thành phần có thể tham gia (Devops, dev,...)





Distributed data



REPLICATION

- Tạo ra nhiều bản sao của dữ liệu trên nhiều node khác nhau.



PARTITION

- Chia dữ liệu ra các phân vùng nhỏ hơn nằm trên các node khác nhau (Sharding)

2

MySQL Replication



Nguyên lý Replication

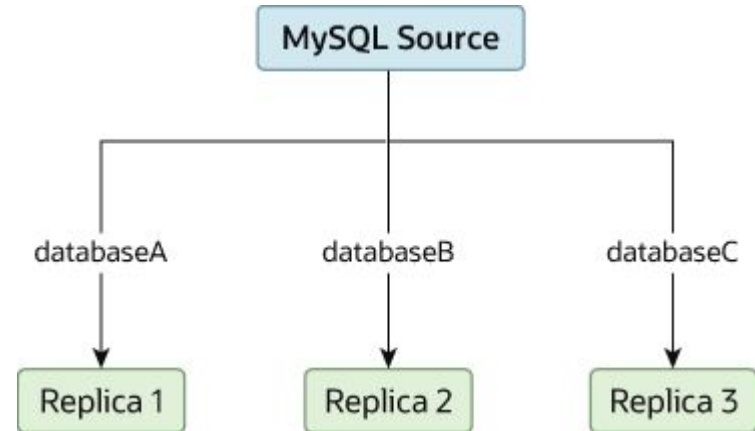


Các mô hình Replication



Định nghĩa

Replication là 1 quá trình cho phép data từ một server MySQL nguồn (*source*) được sao chép tới một hoặc một số MySQL database servers khác (*replicas*).





Các thuật ngữ



Source/Master: Database nguồn

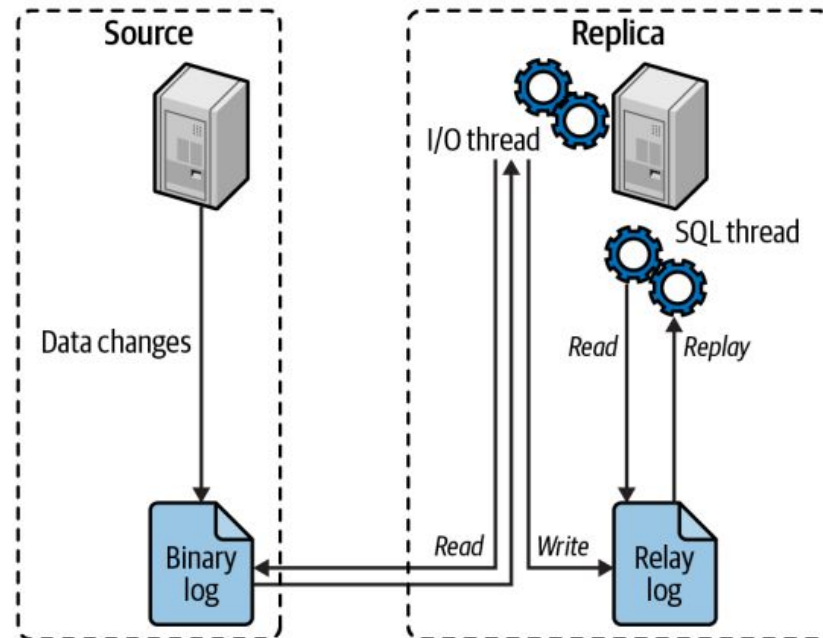


Replica/Slave: Database được nhân bản dữ liệu từ database nguồn.



Binlog: Là một file log được tạo bởi MySQL để các thay đổi của dữ liệu trong CSDL

- Sao chép dữ liệu (replication)
- Khôi phục dữ liệu (data recovery)
- CDC (Capture Data Change)





Ứng dụng của Replication

Giải pháp mở rộng (Scale-out)

Scale read: khả năng read (SELECT) thông qua việc số replicas

Analytics

Truy vấn dạng phân tích xử lý ở Replicas, không ảnh hưởng tới Source

HA & failover

Đảm bảo tính sẵn sàng cao và dự phòng

01

03

05



02

04

06

Data security, Data archiving

Cơ chế backup dữ liệu, lưu dữ liệu ở nhiều nơi khác nhau.

Test MySQL upgrade

Replicas có thể khác version với Source.

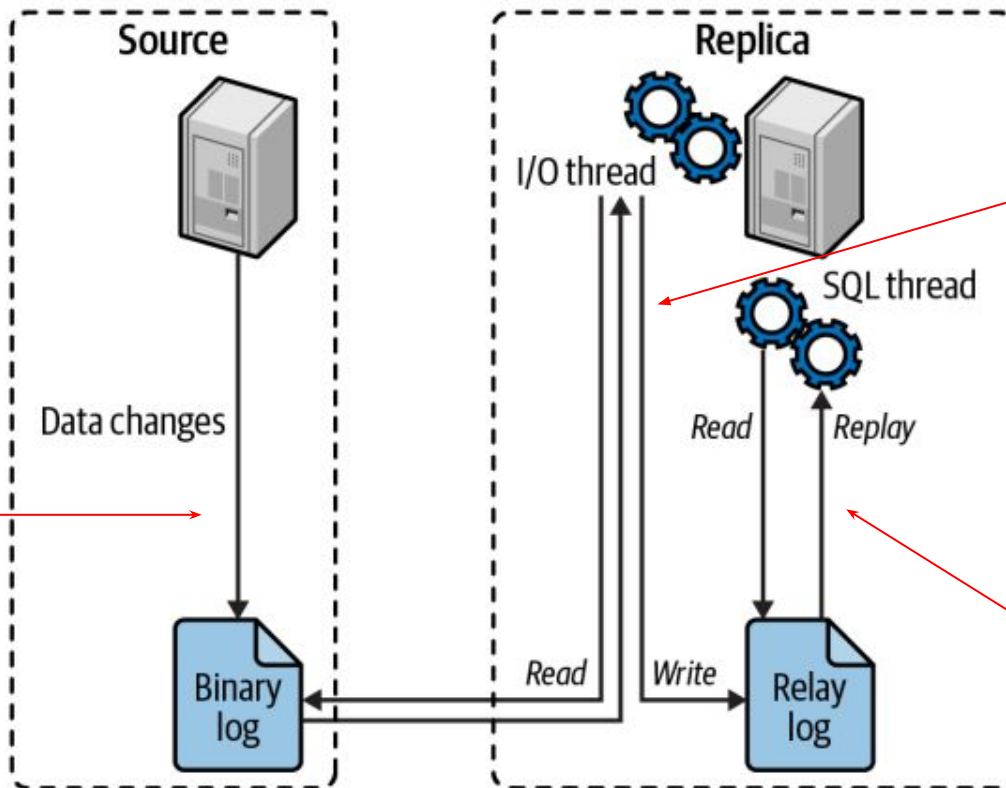
Selective replication

Replicate từng phần dữ liệu cho các replicas khác nhau.
Replicas sẽ tận dụng tốt việc sử dụng memory cho đúng nghiệp vụ nhất định



Nguyên lý replication - 3 phase

(1) **Source create Binlog:** Dữ liệu ở Source thay đổi được lưu vào binary log.



(2) **Replica I/O thread:** Replica tạo I/O thread đọc binlog từ db Source và write xuống Relay log của Replica.

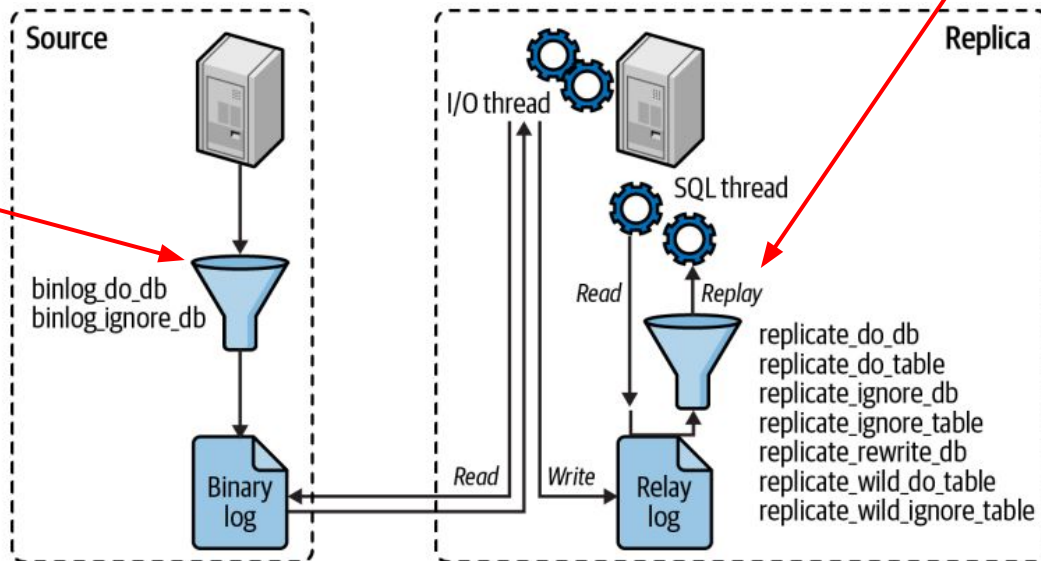
(3) **Replica SQL thread:** Replica tạo SQL thread đọc relay log và apply thay đổi xuống replica.



Replication filtering

Là mô hình GHTK đang ứng dụng cho hệ thống Ecom (Source) -> ERP, OS, CS (Replicas)

Filter phía Source
(Không khuyến khích):
binlog_do_db,
binlog_ignore_db chỉ
định các bảng sẽ được
write/ignore binlog.



Filter phía Replica (Sử dụng hạn chế): Chỉ định các bảng được apply relay log.



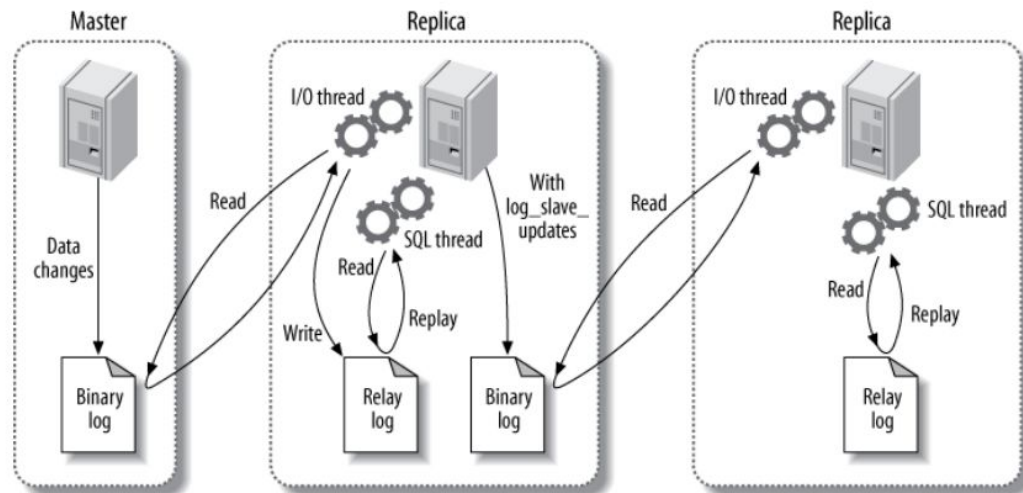
Replication bắc cầu



Replica có thể enable binlog và gửi event binlog đó tới các Replica khác.



Replica có thể làm Source cho các replica khác, write thêm các bảng của riêng nó, miễn sao không xung đột với các bảng của Source (Trường hợp của Ecom ERP)





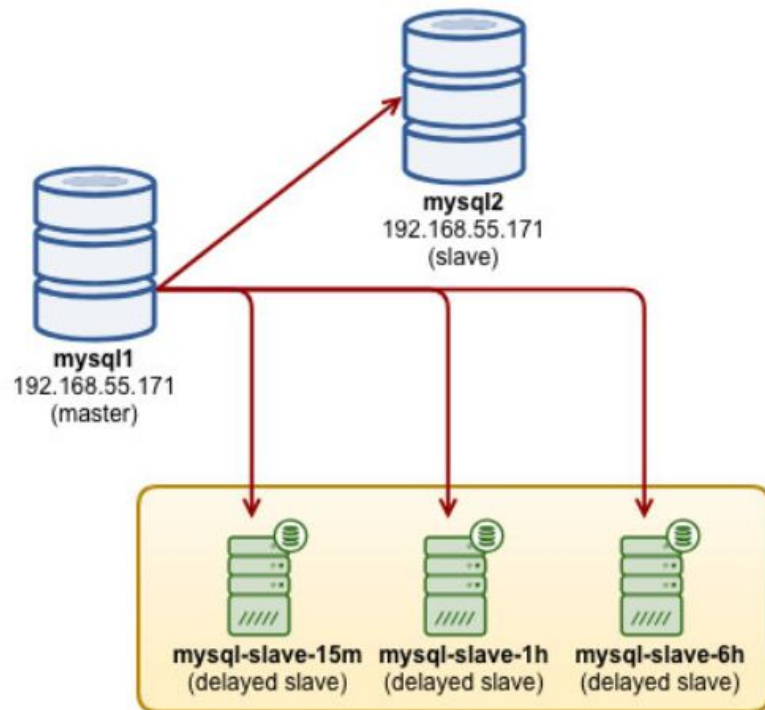
Delayed Replication

Dùng như 1 bản backup dữ liệu, giúp khôi phục dữ liệu khi xảy ra sự cố nghiêm trọng.



Khi có một giao dịch DROP nhầm bảng lớn, logic phức tạp,... => việc khôi phục diễn ra lâu.

Có thể dùng bản backup này loại bỏ transaction gây ra sự cố (qua GTID) và tiếp tục quá trình replication để khôi phục dữ liệu.

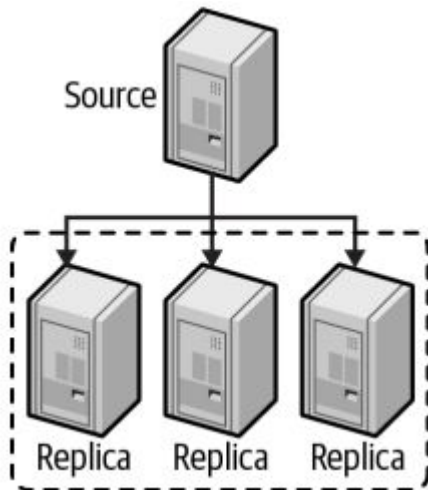




Replication Topologies Basic

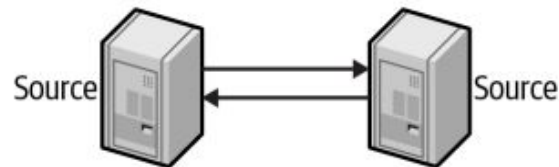
Khuyến khích

Active/Read Pool

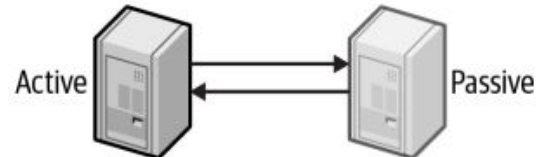


Không khuyến khích

Dual source in active-active mode



Dual source in active-passive mode

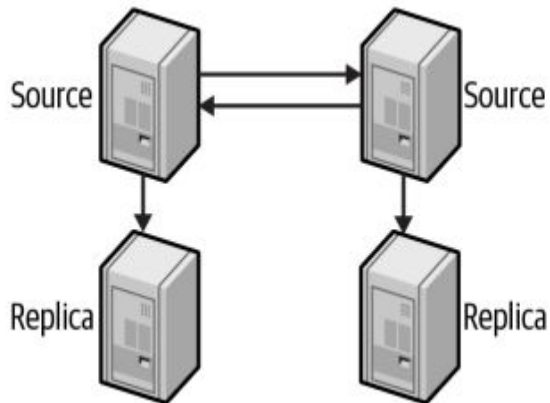




Replication Topologies Basic

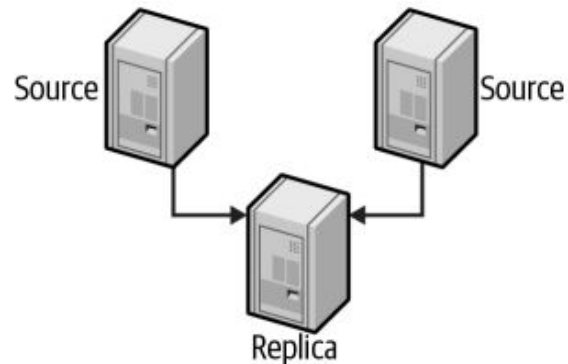
Không khuyến khích

Dual sources with replicas



Không khuyến khích

Multi-source replication





Replication/ Binlog format

Statement-based



Binary log file: Câu lệnh (statement) SQL



Replica thực thi các câu lệnh SQL từ relay log



Ưu điểm: Đơn giản, quá trình transfer binlog nhanh.



Nhược điểm: Không an toàn, gặp vấn đề với các truy vấn không xác định rõ output



Replication/ Binlog format

Row-based (Recommend)

Sử dụng phần lớn trong các mô hình replication ở GHTK



Binary log file: Events bản ghi thay đổi (Before row, after row)



Replicas sao chép các events thay đổi từ Source.



Ưu điểm: An toàn về mặt dữ liệu, áp dụng được các kỹ thuật CDC



Nhược điểm: Binlog size lớn.



Replication/ Binlog format

Mixed



Mặc định statement-based, sẽ chuyển qua row-based trong một số trường hợp



Phụ thuộc câu lệnh, storage engine, cần 1 số điều kiện để apply.

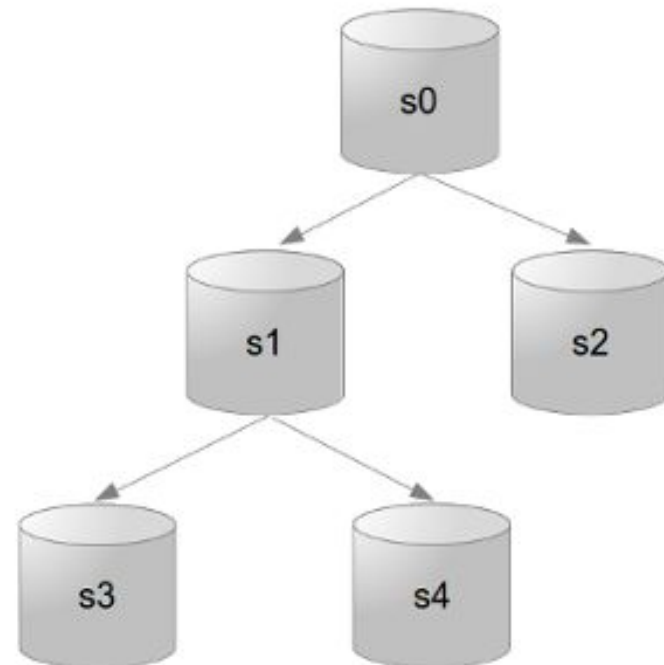


GTID - Global Transaction ID

GTID là định danh của 1 transaction được committed trên server (Source) . Định danh này là duy nhất trên toàn bộ server trong mô hình replication (replication topology).

`GTID = node_id:transaction_id`

- Node ID: ID của máy chủ MySQL đã tạo ra giao dịch. (server_uuid)
- Transaction ID: ID của giao dịch trong phạm vi node đó.





GTID - Tầm quan trọng



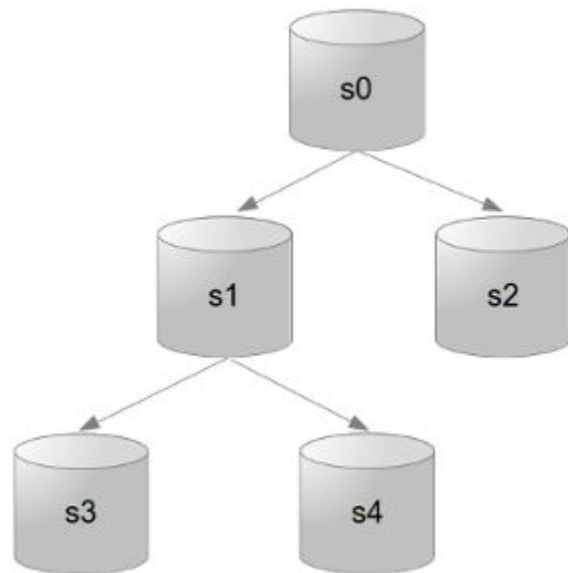
Sao chép dữ liệu và khôi phục dữ liệu dễ dàng.



Xử lý lỗi trong replication khi bị mất đồng



Cho phép sử dụng replication master-master, sửa đổi mô hình replication gốc.

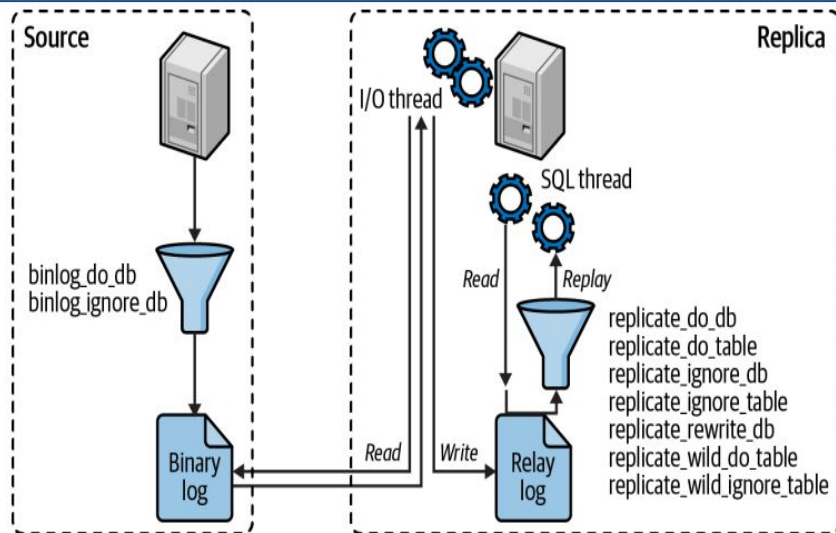




Replication Lag

- Là hiện tượng replica đồng bộ dữ liệu chậm.
- Gần như không thể tránh khỏi trong mô hình Replication (Định lý CAP)

Lag = <Tgian GTID thực thi ở Replica> - <Tgian GTID được sinh ra ở Source>





Replication Lag - Nguyên nhân lag



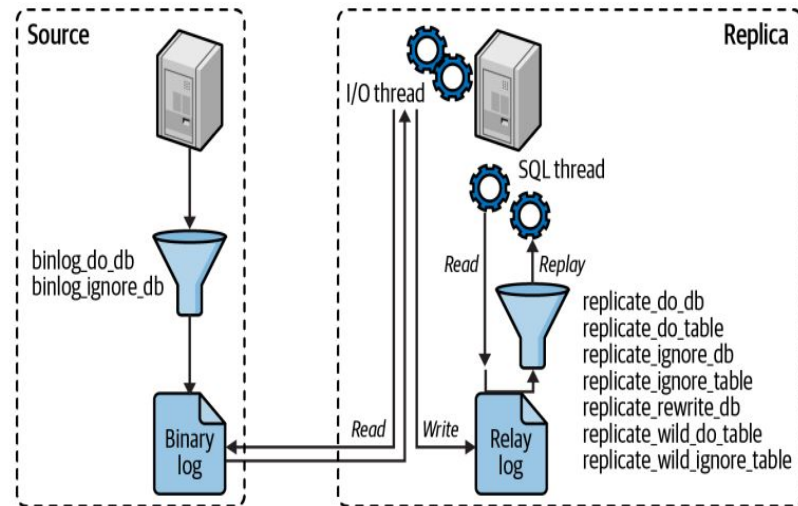
I/O thread kéo binlog về bị chậm:

- Network chậm
- Master cao tải (Slow query, SELECT/INSERT/UPDATE,...)
- Lượng binlog sinh ra từ master quá lớn: Transaction lớn, nhiều transaction update, alter bảng,...



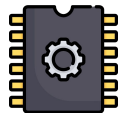
SQL thread apply vào db chậm:

- Luồng read: Replica cao tải, I/O cao do slow query, select nhiều, count/sum/aggregate,...
- Luồng write: Lượng binlog phải apply lớn (toàn bộ replication lag)





Xử lý Replication Lag



Hardware:

- Scale Replicas, write ở Source, read ở các server Replicas, phân tải luồng read, scale multi-layer replication.
- Disk I/O: NVMe, memory -> innodb_buffer_size



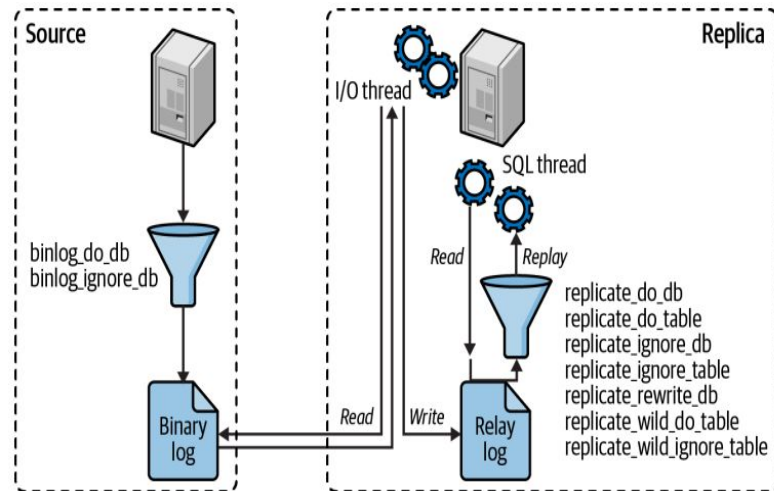
MySQL config:

- Lower durability: sync_binlog=0, innodb_flush_log_at_trx_commit=0
- Multithreaded replication
- Sharding scale write



Software:

- Hạn chế transaction lớn.
- Hạn chế sinh nhiều binlog cùng 1 thời điểm.
- Xử lý slow query, giảm luồng read.
- Giảm luồng write ở Source
- Giảm luồng write ở Replica (replication filter)
- Sống chung với lag!



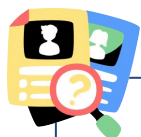


Vấn đề read-after-write



Hiện tượng: Shop đăng đơn báo thành công nhưng lúc vào màn hình đơn hàng lại không thấy đâu

Nguyên nhân: Write ở Source và đọc ở Replica, có hiện tượng replication lag.



Mặc định replication (async) không đáp ứng được tính nhất quán read-after-write.



Solution:

- Chia nhỏ logic, đọc ghi dữ liệu trên cùng 1 node.
- Bắt buộc quá trình replication phải là synchronously
 - *Luồng write phải nhận ACK từ replicas*
 - *Phức tạp trong mô hình, luồng write bị chậm, replicas bị lỗi?*
- Ứng dụng chấp nhận được việc read-after-write in-consistency
 - *Phát hiện được đang có lag, chủ động retry*
 - *Kiểm soát được dữ liệu update-to-date tới đâu là chấp nhận được.*
 - *Sử Dụng Timestamp hoặc Versioning*

Tổng kết

- MySQL replication là tính năng rất quan trọng và được sử dụng rộng rãi. Có thể là nguyên nhân làm MySQL phổ biến.
- Không thể scale replica vô hạn, càng nhiều replica sẽ gây cao tải Source do luồng đọc I/O thread.
- Giữ cho mô hình replication đơn giản nhất để tránh lỗi. Sử dụng thận trọng với các mô hình Active-active, Replication filtering, Replica làm Source cho replica khác,...
- Hiểu và sử dụng Binlog format, GTID.
- Hiểu về cơ chế replication. Giám sát replication lag và cách xử lý các tình huống lag.

3

**MySQL
Scaling**

Tổng quan về Scaling



**Các mô hình scale MySQL
tại GHTK**

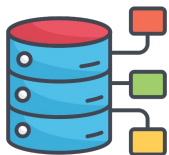
Scaling Overview



Hãy nghĩ về data system...

Chuyện gì sẽ xảy ra khi traffic tăng, dữ liệu tăng?

Hệ thống có thể mở rộng bằng cách tăng tài nguyên?



Database Ecom GHTK:

- 2022: 17TB
- 2023: 24TB (Tăng 40%)



Trịnh Đình Biên

Năm nào campaign mình cũng phải add thêm resource cho cả dbreport và web admin/cs mà số đơn không tăng theo. Là do nghiệp vụ tăng nhiều quá, hay do càng ngày ứng dụng của mình performance càng tệ ae?



Nguyễn Thế Huy

66

Năm nào campaign mình cũng phải add thêm resource cho cả dbreport và web admin/cs mà số đơn không tăng theo. Là do nghiệp vụ tăng nhiều quá, hay do càng ngày ứng dụng của mình performance càng tệ ae?

Trịnh Đình Biên, 11:59 14/11/2023

logic nó cũng phức tạp tăng dần đều mà anh 😊

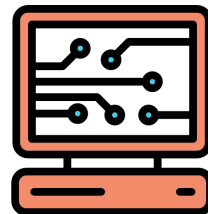


Scaling Overview



Khả năng mở rộng: Hệ thống có thể đáp ứng được nhu cầu khi traffic tăng, tải tăng mà không làm giảm hiệu năng.

- **Vertical Scaling** (Mở rộng theo chiều dọc): Tăng tài nguyên cho 1 máy chủ (Thêm disk, ram, cpu,...)
- **Horizontal Scaling** (Mở rộng theo chiều ngang): Thêm nhiều máy chủ



Xác định workload của ứng dụng:

- **Read-heavy:** Ứng dụng chủ yếu sử dụng luồng đọc -> Scale read
 - Sử dụng các mô hình replication
- **Write-heavy:** Ứng dụng nặng ở luồng ghi -> scale write
 - Database Sharding
 - Functional Sharding: Tách bảng, tách logic, nghiệp vụ.



Các mô hình scale MySQL

Tách luồng read-write, scale read:

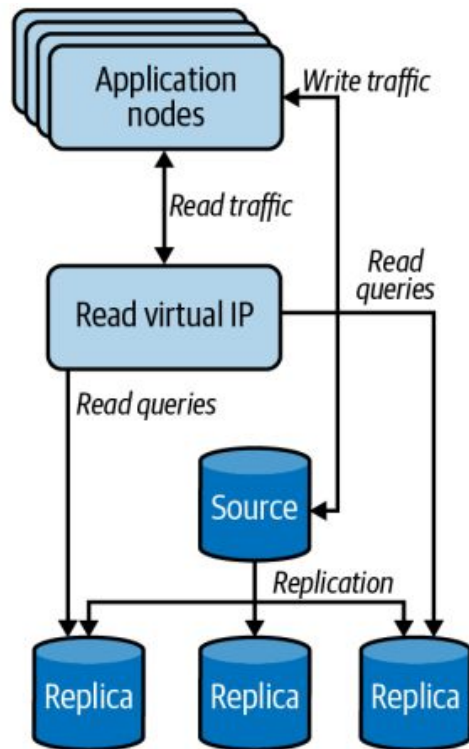
- Write traffic đổ vào db Source
- Read traffic đi qua Load Balance tới các Replicas. Cấu hình phân tải ở LB.
 - Random, Round Robin
 - Fastest response
 - Weighted,...



Ưu điểm: Đơn giản, dễ triển khai, áp dụng cho mô hình replication truyền thống tại GHTK.



Nhược điểm: Giới hạn scale luồng read, số lượng replica có hạn vì có thể gây tăng tải DB Source.



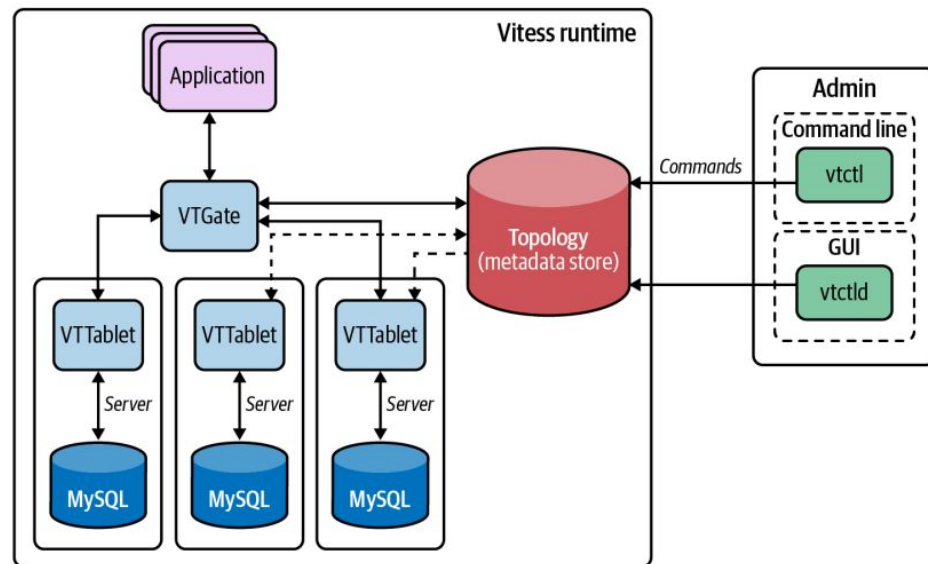


Các mô hình scale MySQL

Scale write:

- Vitess database-clustering system for MySQL
- Sharding

Chưa thử nghiệm và chưa được sử dụng ở GHTK

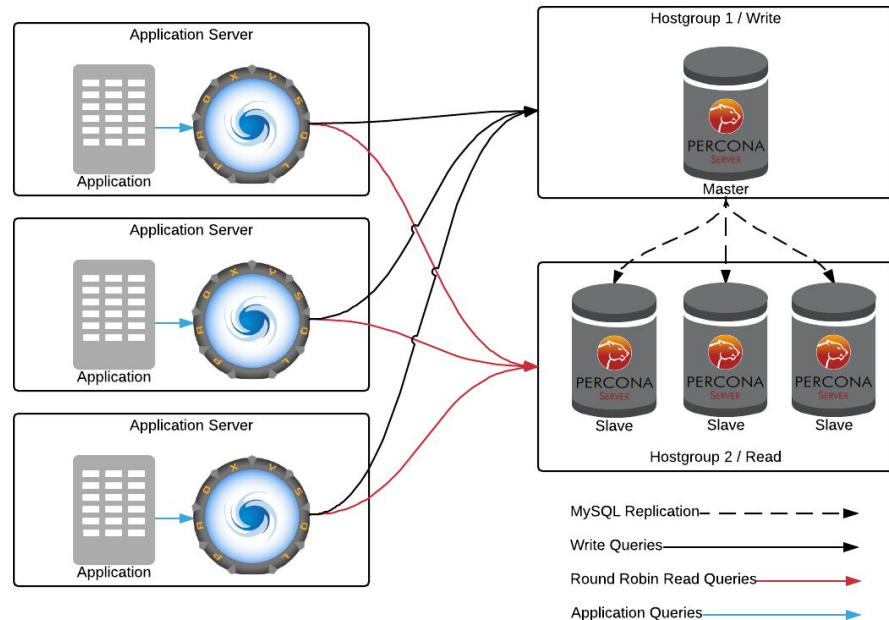




Các mô hình scale MySQL

ProxySQL:

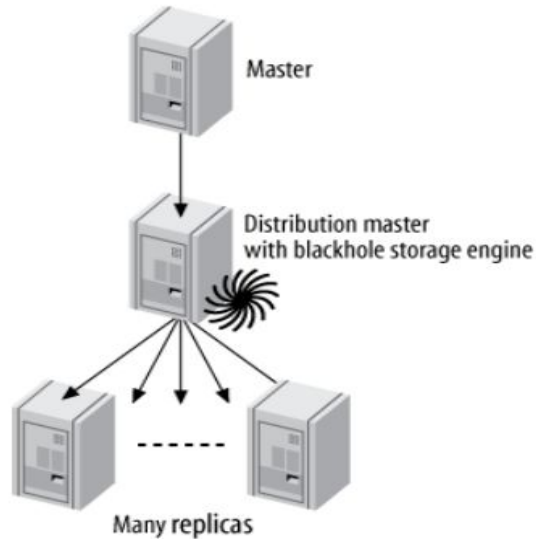
- Load Balancing
- Khả năng chịu lỗi: Ngắt kết nối khỏi db bị lỗi.
- Connection management, connection pool.
- Caching Query Results
- Query Rewriting





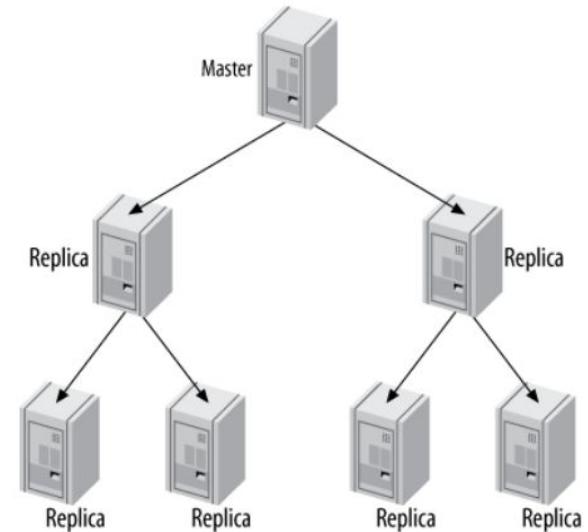
Các mô hình scale MySQL

Distribution master,
many replicas



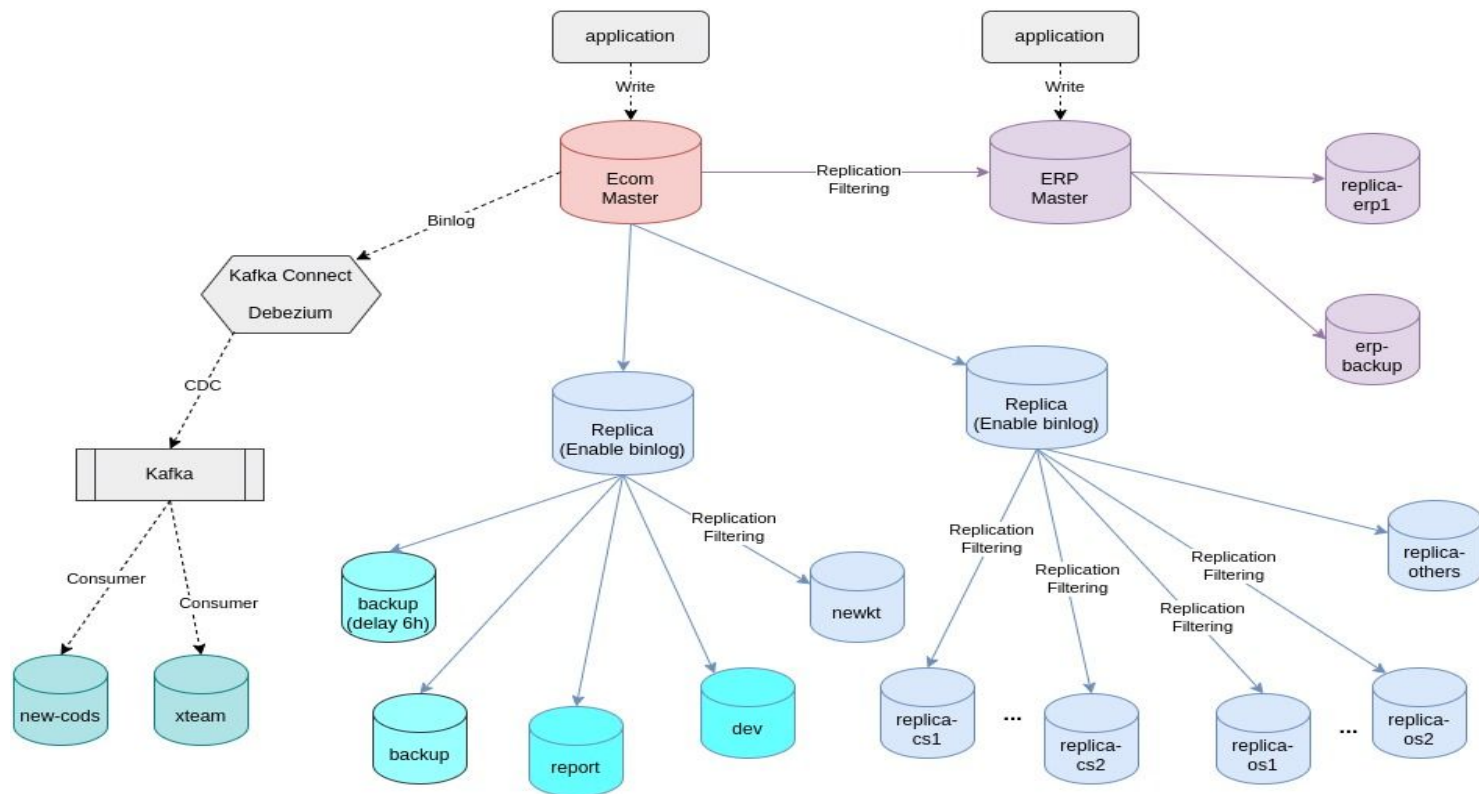
Replication Multilayer Topologies

Pyramid replication





Scale Ecom tại GHTK



TỔNG KẾT



Khi xây dựng 1 hệ thống data system cần quan tâm tới độ tin cậy, khả năng mở rộng và khả năng bảo trì.



Replication MySQL là một tính năng quan trọng, cần hiểu rõ cơ chế và phương án xử lý các vấn đề của Replication.



Có nhiều phương án Scale MySQL, tùy theo nhu cầu và quy mô ứng dụng để có hướng scale phù hợp.



LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

TRÂN TRỌNG CẢM ƠN

Chúc bạn có trải nghiệm học tập và làm việc tốt tại GHTK

